Lab #3

Write a main program and a subroutine using PIC18F assembly language.

ECE 3301L Introduction to Microcontrollers Laboratory

Sarah Boyd

6/30/25

Dr. Tamer Omar

Cal Poly Pomona

College of Engineering

Execution videos and program files:

Prelab :

- General code explanation: https://youtu.be/g594kihXLBI
- STKPTR explanation: https://youtu.be/FInXINhOUpo

Postlab

- Output analysis explanation: https://youtu.be/novM-mS_zoM

Objective: The purpose of this lab is to:

- write a main program and a subroutine in PIC18F assembly language.
- assemble and debug the main program and the subroutine using Microchip's MBLAB assembler/debugger.
- demonstrate using the MPLAB how the hardware stack pointer (STKPTR) changes with the execution of the PIC18F CALL and RETURN instructions.

PRELAB:

---

prelab i)

It is desired to write a subroutine in PIC18F assembly language to compute $Z = \sum_{i=1}^{8} X_i$

Assume the Xi's are unsigned 8-bit and stored in consecutive locations starting at 0x50. Assume FSR1 points to the Xi's. Also, write the main program in PIC18F assembly language to perform all initializations (FSR1 to 0x50, STKPTR to 5), call the subroutine, and then compute Z/8. Discard the remainder.

(a) Flowchart the problem.

(b) Convert the flowchart to PIC18F assembly language program

Conceptual background and explanation of code:

First, we need to understand the meaning of the subroutine. The subroutine is $Z = \sum_{i=1}^{8} X_i$. This equation means that we have 8 values (X_1 through X_8) and the result (Z) is the sum of all of the values together. After calling the subroutine, we are instructed to divide Z by 8 and discard the remainder.

We have 8 variables (X_1 through X_8) and each variable is 8 bits, meaning it can hold a value from 0 to 255. Each variable is also unsigned, meaning that they can't be negative. The program needs to read the 8 bytes from memory starting at 0x50 (stored in RAM) in consecutive memory locations.

Variable	Location	Expected value
X1	0x50	2
X2	0x51	3
X3	0x52	4
X4	0x53	5
X5	0x54	6
X6	0x55	7
X7	0x56	8
X8	0x57	9

Variable	Location	Purpose
Result	0x21	Holds result of summing after loop, which is 44
Counter	0x60	Loop counter -holds value 8 to 0, starts at 8 but ends at 0.
Z	0x21	Result / 8 (without remainder), which is 5

Table 1 and 2. *Meaning of memory locations for the prelab program*

In the instructions, it is mentioned that we are supposed to “assume FSR1 points to the Xi’s”. Recall that the FSR1 is a pointer that is used to access memory, and stands for File Select Register. The PIC18F has three indirect addressing registers- FSR0, FSR1, and FSR2. Each register is split into two 8 bit registers. FSR1 is a 12 bit register made up of two 8- bit SFRs - FSR1H (upper byte) and FSR1L (lower byte). In this code we use POSTINC1 to access FSR1 and then increment it to point to the next item on the stack.

To divide the sum (Result) by 8 to get Z, we can use right shifts. The PIC18F does not have a divide instruction, but dividing by 2 can be achieved by using right shifts - each right shift by 1 bit divides by 2.

Shift right once -> divide by 2

Shift right twice -> divide by 4 (2^2)

Shift right thrice -> divide by 8 (2^3)

We use the command RRCF (Rotate Right through Carry Flag) to do this. First, the LSB (least significant bit) of the register is moved into the carry flag. Then, the carry flag moves into the MSB (most significant bit) of the register and all other bits shift right by one position. For example, we start with 1010 1000 (168). The LSB is 0, and is moved into the LSB position now. Shifting the bit one to the right gives us 0101 0100 or 84, and 84 is equivalent to half of 168. Before each RRCF, we clear the carry flag by doing ADDLW 0 to make sure that the LSB position of the register is not from a previous operation. The remainder naturally is dropped and

lost after doing bit shifts, which conveniently corresponds with the guidelines for writing the prelab code. Note that we are using the RRCF command three times to divide by 8. The process for getting Result =5 is shown in more detail below.

First, Result = 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 44.

44	0 +	0 +	32 +	0 +	8 +	4 +	0 +	0
Place	$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Bit	0	0	1	0	1	1	0	0

Table 3. Decimal value of 44 in terms of binary value

Carry is initially cleared to 0, so we start with C=0.

Result = 2C (hex for 44)
0010 1100 (44)
LSB = 0
C=LSB =0
MSB= previous carry =0
0001 0110 (22)

Result = 16 (hex for 22)
0001 0110 (22)
LSB = 0
C=LSB=0
MSB = previous carry= 0
0000 1011 (11)

Result = B (hex for 11)
0000 1011 (11)
LSB =1
C=1
MS= previous carry =0
0000 0101 (5)

Result = 5

As a review, the STKPTR is the stack pointer for the instructions for the PIC18F. The stack pointer register is a special function register crucial for managing the stack and is especially relevant for subroutine execution. This whole process of managing the order of the program is already in the hardware and the stack pointer doesn't typically have to be set to a value, but for demonstration purposes it is set to 5 in this prelab code. One of the goals of this prelab is to see how the stack pointer changes when a subroutine is included, and this is explained in the second prelab video titled "STKPTR explanation" listed on page 2.

The stack pointer holds the memory address of the current top of the stack, and the stack pointer register is really important for managing the stack during operations like calls and returns. The stack pointer register ensures that the program follows the correct sequence for executing instructions even when a subroutine is included in the code. The hardware stack is used to keep track of the return addresses when the subroutines are called, and allows the program to return back to the correct location after the subroutine is executed and complete. So when the subroutine is called, the return address is pushed onto the hardware stack and the program counter is updated to jump to the subroutine. When the subroutine returns (the RETURN instruction is executed), the return address is popped from the stack and the program execution continues back as normal.

In line 39, we set WREG equal to 5 and then set the stack pointer equal to WREG, giving it the value 5. So before the CALL instruction, STKPTR equals 5.Next, after doing CALL SUM, STKPTR auto increments to 6. When the subroutine is executed, the counter eventually reaches 0 and the subroutine is completed. The RETURN instruction is reached, and the top address is popped off the stack and the stack pointer automatically de increments back to 5. The stack pointer returns to the value it was set as before the call (5), and thus we jump back to the instruction after CALL.

Typically in my programs I use the command HERE: BRA HERE to create an infinite loop and then END to tell the assembler my program is done. However, for this particular program, using these instructions caused my program to not work. Using the instruction SLEEP fixed my issue. The SLEEP instruction stops the program from running and helps it be more "stable", leaving the

registers untouched and letting the values remain. The instruction BRA HERE makes the CPU keep fetching and executing instructions, which might be causing issues.

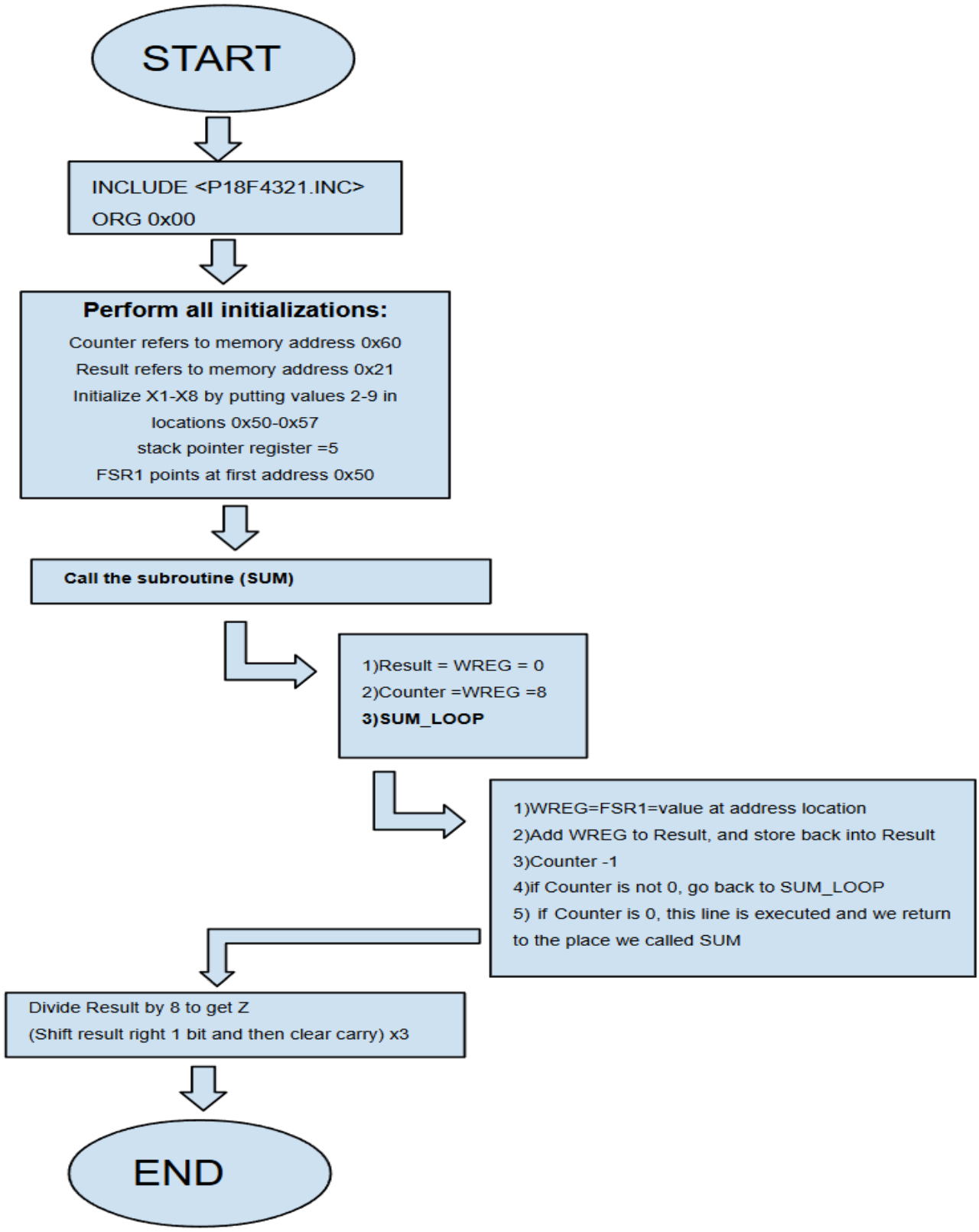
Here are some instructions and directives that will be used in my prelab code.

Instruction	Function
MOVLW	Move literal value into WREG
MOVWF	Move WREG into file register
MOVF	Move value from the file register into WREG
SWAPF	Swap upper and lower 4 bits of a byte
ANDLW	Mask off bits you don't want, and used to keep just part of a byte (the high nibble or low nibble)
IORWF	Combines two binary values (the value in the WREG register and the value in a file register) and then stores result back in file register if ", F" or back into WREG if there's a ",W" at the end
SUBWF	Subtracts WREG a file register.
ADDWF	Adds file register to WREG
POSTINC1	First, we read the value of the memory address that FSR1 points to. FSR1 = memory address = value at that memory address. That value is moved into WREG. After reading the value, the pointer FSR1 is incremented by 1, and points to the next memory address. After execution, WREG holds the value and FSR1 points to the next memory location.
POSTDEC1	Read value at address and then deincrement FSR1
LFSR	(Load File Select Register), load a 12 bit memory address into one of the file select registers (FSRs), such as FSR1 (
CALL	Used to jump to subroutines
CLRWDT	Good to include every once in a while in long loops, delays and places where the code takes a while to finish to prevent unwanted resets of your

	program. This command resets the watchdog timer count to 0 and clears the TO and PD in the STATUS register.
CLRW	Clear WREG
RRCF	“Roatate Right through Carry Flag”
ADDLW 0	Used to clear the carry flag, for before shifting the bits right.
GOTO	Jump to label (START)
SLEEP	Halts program from executing any more instructions

Table 4. *Instructions used and their meaning.*

prelab a) Flowchart the problem


START
INCLUDE <P18F4321.INC>
ORG 0x00
Perform all initializations:
Counter refers to memory address 0x60
Result refers to memory address 0x21
Initialize X1-X8 by putting values 2-9 in
locations 0x50-0x57
stack pointer register =5
FSR1 points at first address 0x50
Call the subroutine (SUM)
1)Result = WREG = 0
2)Counter =WREG =8
3)SUM_LOOP
1)WREG=FSR1=value at address location
2)Add WREG to Result, and store back into Result
3)Counter -1
4)if Counter is not 0, go back to SUM_LOOP
5) if Counter is 0, this line is executed and we return
to the place we called SUM
Divide Result by 8 to get Z
(Shift result right 1 bit and then clear carry) x3
END

prelab b)

Convert the flowchart to PIC18F assembly language program

Asm Source | History

```
1   #include <P18F4321.INC>
2
3   ;Symbolic names for memory locations
4   Counter EQU 0x60      ; Counter variable value for the loop in the subroutine is st
5   Result  EQU 0x21      ; The result of summing is stored at memory location 0x21
6
7
8     ;Main program
9       ORG 0x00          ; Program starts at address 0
10
11  ; First, we need to initialize the memory locations of X1-X8
12  ; We do this by storing 8 values (representing X1 through X8) at locations 0x50-0x
13  ; Load literal 2 into WREG, and then store it in RAM address 0x50
14      MOVLW   2           ;WREG = 2
15      MOVWF   0x50        ;0x50 holds the value 2 (X1)
16  ;Load literal 3 into WREG and store it at 0x51
```

Output | Debugge... | Search R... | File Regi... | Program... | SFRs | Variables | Notificat... | Call Stack

Name	Type	Address	Value
0x50	(1) Bytes	0x50	0x02
0x51	(1) Bytes	0x51	0x03
0x52	(1) Bytes	0x52	0x04
0x53	(1) Bytes	0x53	0x05
0x54	(1) Bytes	0x54	0x06
0x55	(1) Bytes	0x55	0x07
0x56	(1) Bytes	0x56	0x08
0x57	(1) Bytes	0x57	0x09
WREG	SFR	0xFE8	0x09
0x21	(1) Bytes	0x21	0x05
0x60	(1) Bytes	0x60	0x00
<Enter new watch>			

IMG 1. *Resulting memory locations have the predicted results. The program is successful. For an explanation of the prelab code, please see the included video on page 2 and the commented code, submitted on Canvas.*

4. Equipment, Software and Components required:

Microchip's MPLAB assembler /Debugger

5.Description (corresponding topics covered in the textbook)

This lab utilizes a pointer, FSR1 to point to Xi's. A subroutine will be written in PIC18F assembly language which will use a loop to compute the summation of 8 numbers. The main program will also be written in PIC18F assembly language which will use the hardware the stack pointer (STKPTR) to call the subroutine. (Example 7.2, Pages 163-167, Problem 7.16, Page 184)

6.Prerequisites:

Stack Pointer register, Subroutine Calls in assembly language, Section 7.4 , Section 7.6

7. Procedure:

i) Assemble and verify the PIC18F assembly language programs for the main program and the subroutine of part (b) using the MPLAB.

ii) Demonstrate using the MPLAB how the hardware stack pointer (STKPTR) changes with the execution of the PIC18F CALL and RETURN instructions.

8. Deliverables:

Postlab

Write a subroutine in PIC18F assembly language at address 0x200 to compute (X^4 /4) where X is an unsigned 8-bit number. Also, write the main program at address 0x100 in PIC18F assembly language that will initialize FSR0 to 0x0070, X is to arbitrary data, initialize STKPTR to 0x10, call the subroutine to compute (X /4), and then push 8-bit result onto the software stack pointer pointed to by FSR0

POSTLAB:

Conceptual background and explanation of code:

The main program should be written at address 0x100, and the subroutine at 0x200. X is arbitrarily chosen to be 5, the stack pointer is initialized to 0x10

FSR0 is initialized to 0x0070, X is arbitrary data, STKPTR is initialized to 0x10

In the postlab we write a subroutine (located at address 0x200). The subroutine aims to compute $\frac{X^4}{4}$, where X is an unsigned 8 bit number. There is no exponential instruction for the PIC18F, so we can use the multiply instruction MULWF (multiply WREG by a file register). The 16 bit result of the subroutine is stored in PRODH (high byte, higher 8 bits) and in PRODL (lower 8 bits). When we have our final high byte and low byte value, we divide them by 4 to get our final anwer. We are only instructed to keep the 8 bit result (only PRODL) and to

Before our subroutine, we start with X= 0x05 or 5 in decimal, and X has the value 5 for the entire program. WREG is set equal to X, and starts with the value 5.

First MULWF

GOAL: to get X^2 by X * X

WREG = X = 5

WREG * X = 5* 5 =25, or 19 in hex

Stored automatically in PRODH= 0x00, PRODL =0x19

Second MULWF

GOAL: to get X^3 by doing X^2 * X

WREG = PRODL = value 25

WREG * X = 25 * 5= 125, or 0x7D in hex

Stored automatically in PRODH= 0x00, PRODL = 0x7D

Third MULWF

GOAL: to get X^4 by X^3 * X

WREG= PRODL=125

WREG * X =125 * 5 = 625, or 0x271 in hex

Stored automatically in PRODH =0x02, PRODL = 0x71

The sum is greater than 255, and so we should use both PRODH and PRODL to adjust for having a higher and lower byte. To perform division by 4, we move one bit to the right two times. The concepts of this process are similar to the division loop in the prelab.

PRODH				
	Hex	Binary	Decimal	Carry/ process
Initially	0x02	0000 0010	2	1)C= 0, cleared 2)LSB =0 3) C =LSB = 0 4)bits shift right 1 and previous carry (0) goes into MSB, giving us 0000 0001
After first RRCF	0x01	0000 0001	1	1)C=0 2)LSB=1 3)C=LSB=1 4) bits shift right 2, and previous carry (0) goes into MSB, giving us 0000 0000
After second RRCF (final value)	0x00	0000 0000	0	1)C=1

Table 5. *Logistics of dividing the high byte*

PRODL				

	Hex	Binary	Decimal	Carry/ Process
Initially	0x71	0111 0001	113	1)C= 0, cleared 2)LSB =1 3) C =LSB = 1 4)bits shift right 1 and previous carry (0) goes into MSB, giving us 0011 1000
After first RRCF	0x38	0011 1000	56	1)C=1 2)LSB=0 3)C=LSB=0 4) bits shift right 2, and previous carry (1) goes into MSB, giving us 1001 1100
After second RRCF (final value)	0x9C	1001 1100	156	1)C=0

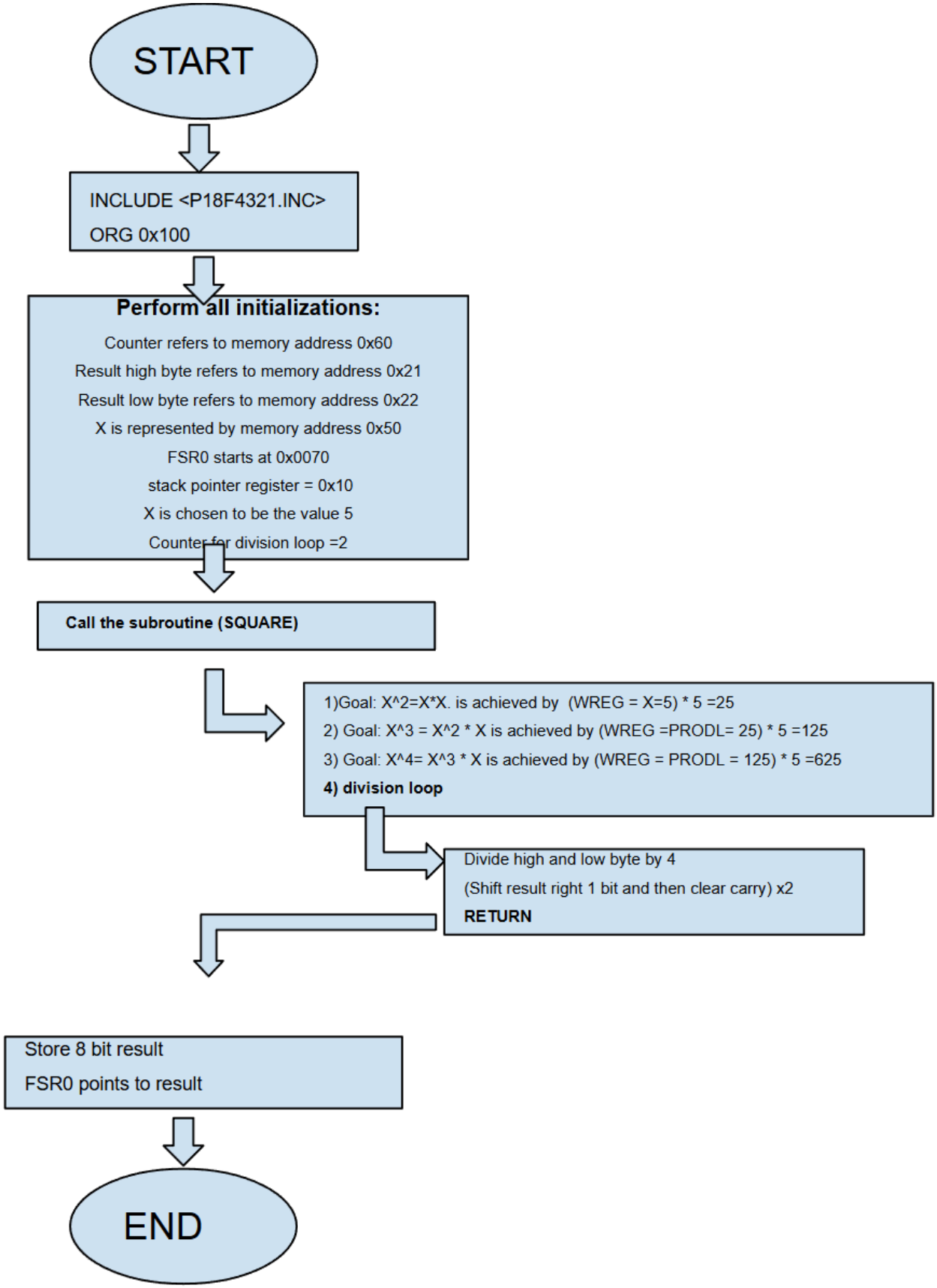
Table 6. *Logistics behind dividing the low byte*

After dividing by 4, we only care about pushing the lower 8 bit result to FSR0, according to the postlab instructions.

Here are the expected results of particular memory addresses:

Variable	Memory address	Expected value
STKPTR	0x10, 0x11, 0x10	Initially 0x10 at line 15, before the subroutine is called, at line 21 after the subroutine is called it will have the value 0x11, and then after return instruction should go back to the value 0x10
ResultH	0x21	High byte of result (PRODH), ends at 0 due to only the lower 8 bits being stored. Both PRODH and ResultH should end with 0x00
ResultL	0x22	Low byte of result (PRODL) should be 156 or 0x9C. Both PRODL and ResultL should end with 0x9C.
Counter	0x60	Initially 2 (because we want 2 iterations). Should end with the value 0, meaning that the division loop is completed.
X	0x50	0x05
FSR0	0x71	0x71 holds the value 0x9C, so FSR0 holds the value of the 8 bit result.
WREG		Before 1st MULWF- 0x05 Before 2nd MULWF -0x19 Before 3rd MULWF- 0x271 End of program- 0x02

Table 7. *Predicted results of some key memory locations*


START
INCLUDE <P18F4321.INC>
ORG 0x100
Perform all initializations:
Counter refers to memory address 0x60
Result high byte refers to memory address 0x21
Result low byte refers to memory address 0x22
X is represented by memory address 0x50
FSR0 starts at 0x0070
stack pointer register = 0x10
X is chosen to be the value 5
Counter for division loop =2
Call the subroutine (SQUARE)
1)Goal: X^2=X*X. is achieved by (WREG = X=5) * 5 =25
2) Goal: X^3 = X^2 * X is achieved by (WREG =PRODL= 25) * 5 =125
3) Goal: X^4= X^3 * X is achieved by (WREG = PRODL = 125) * 5 =625
4) division loop
Divide high and low byte by 4
(Shift result right 1 bit and then clear carry) x2
RETURN
Store 8 bit result
FSR0 points to result
END

```
25      MOVFF   PRODH, ResultH ;Copy PRODH into ResultH, 0x00
26
27  ; Push only the lower 8 bits result onto software stack at FSR0, in accordance to the instructions
28      MOVFF   ResultL, PREINC0  ;Move low byte result to location pointed by FSR0, and then increment FSR0
29
30
```

Output | Debugger Console | Search Results | File Registers | Variables | Program Memory | SFRs

Name	Type	Address	Value
WREG	SFR	0xFE8	0x02
STKPTR	SFR	0xFFC	0x10
0x21	(1) Bytes	0x21	0x00
0x60	(1) Bytes	0x60	0x00
0x50	(1) Bytes	0x50	0x05
PRODH	SFR	0xFF4	0x00
PRODL	SFR	0xFF3	0x9C
FSR0	SFR	0xFE9	0x0071
0x22	(1) Bytes	0x22	0x9C
0x71	(1) Bytes	0x71	0x9C
<Enter new watch>			

IMG 2. *Resulting values of the variables in Postlab Lab 3. Actual results match predicted results exactly.*

Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E
000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
020	00	00	9C	00	00	00	00	00	00	00	00	00	00	00	00
030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
050	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00
060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
070	00	9C	00	00	00	00	00	00	00	00	00	00	00	00	00
080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

IMG 3. *Resulting values at memory locations in Postlab Lab 3. Actual results match predicted results exactly.*